Using fewer tokens without losing answer quality

Short technical exercise for the internship 2026-INT-298
Farah Belghith, ENSIT Tunis · November 2026
Code, data and full answers: https://github.com/Farah-Belghith2710/token-efficient-assistant

Why I chose this topic. Last summer, during my internship at TDS, I built a RAG chatbot [1] for an Industry 4.0 lab. It used Groq's free plan. After testing it many times, I reached the daily limit of 100,000 tokens. This happened even though each question used only about 2% of the model's context window. I realised that the main issue was not the size of the context window. It was the number of tokens used across requests. At the time, I only estimated token use by dividing the number of characters by four. For this exercise, I wanted to measure it more accurately using the counts returned by the API, while also checking that the answers stayed correct.

A. Five ways to use fewer tokens

Each assistant request contains several parts: instructions, chat history, passages found by search, the user's question and the answer. I grouped the ideas below by the part they help reduce.

Approach	How it works	Why it helps	What can go wrong
1. Retrieve fewer, better passages	Return fewer sections from search (a smaller top-k), remove weak matches, or place the most relevant sections first.	Reduces input tokens. Retrieved passages are often the largest part of the prompt. Removing irrelevant text can also help the model focus [2].	The search may drop the exact passage needed. The assistant could then give a wrong answer or say it does not know.
2. Shorten the passages	Keep only the sentences that matter for the question, or use a smaller model to remove less useful text. Examples include LLMLingua [3] and RECOMP [4].	Reduces input tokens, especially for long pages, tutorials, logs and tables.	The extra compression step also takes time and tokens. It may remove an important number or condition, and exact source citations can become harder.
3. Keep chat history short	Send only the latest messages, summarise older parts [5], or rewrite a follow-up question so it makes sense on its own.	Stops input token use from growing with every message in a long conversation.	A summary may leave out a detail the user refers to later, such as "and the second one?".
4. Use shorter, reusable instructions	Shorten fixed instructions, put them first so Groq can cache them [6], answer simple greetings with fixed rules, or send easy questions to a smaller model [7].	Can lower the cost of repeated requests. On Groq, cached tokens cost half as much and do not count towards rate limits [6].	Shorter instructions may make citations or refusals less reliable. Caching lowers the price, not the number of tokens, and a model router can choose the wrong model.
5. Control the answer and reasoning	Ask for short answers with sources, use a lower reasoning effort when suitable, and set a maximum output-token limit.	Reduces output tokens, including hidden reasoning. Output tokens are slower and cost four times as much as input tokens for gpt-oss-20b on Groq [8].	Answers may become too short for difficult questions. Hidden reasoning also counts towards the limit, so a low limit can cut off an answer.

A note from my internship. At TDS, I was already using part of idea 4 without knowing its name. Greetings, contact details and the list of lab modules were handled by fixed rules, so these requests did not need to reach the LLM.

B. The two ideas I tested first

I chose idea 1 (retrieve fewer passages) and idea 5 (control the answer) for three reasons:

- **They target the biggest costs.** In my baseline, retrieved passages made up 79% of the prompt text. On the output side, gpt-oss-20b also uses hidden reasoning tokens before giving an answer, and those tokens count too.
- **They are easy to test separately.** Each idea mainly changes one setting, so I can see what caused a change in quality. Chat memory (idea 3) matters more in long conversations, while shortening passages (idea 2) needs an extra model. I left those for later.
- **They have different risks.** Retrieving fewer passages can remove an important fact, while limiting the answer can leave out part of the response. Testing both helps show these two risks.

How I tested idea 1: fewer passages. The setting I changed was k, which is the number of sections returned by the search. Before calling the LLM, I checked each value from k=1 to k=8. For each of the 24 answerable questions, I checked whether the retrieved text contained all the facts needed for a complete answer. This check was free because it did not require any LLM calls.

k value	1	2 to 4	5 or 6	7 or 8
Questions fully covered	58%	92%	96%	100%

I chose k=5 because it was the smallest value that kept 23 of the 24 questions answerable. I also tested k=3 to see what would happen with an even smaller result set. My first version of the check only looked at whether the correct document was

found. That gave a much higher score (100% from k=4), but finding a document does not guarantee that the search found the exact section containing the answer. I therefore changed the check to look for all the required facts.

How I tested idea 5: shorter answers and less reasoning. I kept the original prompt and added one instruction: "At most 3 short sentences, no introduction." I also changed the reasoning effort from medium to low and reduced the output-token limit from 2,048 to 400.

What I measured. For each answer, I recorded the exact input, output and reasoning token counts returned in Groq's `usage` field, as well as the response time. I also checked answer correctness (whether all key facts were included), citations (whether the right documents were named), refusals on the 6 questions the documents could not answer, incorrect refusals, and whether any answer was cut off by the token limit.

How I judged whether quality had dropped. I set the pass rule before running the tests. The optimized agent would pass if it made **no more than one extra mistake** compared with the baseline (1 out of 24 questions, or about 4 percentage points), refused at least as many unanswerable questions, and never had an answer cut off by the limit. I used a temperature of 0.6, within Groq's recommended range of 0.5 to 0.7 for reasoning models [9]. This means answers can vary slightly between runs. I therefore ran the optimized agent twice to see how much the results changed from one run to another. I also read every answer whose score changed, so I was not relying only on the automatic check.

C. The experiment: baseline vs. optimized agent

Test setup. I created 16 short documents for a fictional research lab called "Kestrel Lab". They included wiki pages, Teams chat exports, tutorials, rules and release notes, to represent the different types of information found in a real organisation. I used AI to help draft the documents, so I knew the correct answer to each question. I split the documents into 53 sections, with one section for each heading. For search, I used TF-IDF with cosine similarity [10]. It is simple and does not require downloading another model. I used gpt-oss-20b [11] through Groq's free plan [12]. The Llama models I had used at TDS were removed from the free plan in August 2026 [13].

The test contained 30 questions: 20 could be answered using one document, 4 needed two documents, and 6 could not be answered from the documents. Some questions were designed to check whether the assistant would make things up. For example, the guest Wi-Fi was mentioned, but its password was not included. The **baseline** used k=8, normal answers, medium reasoning and a 2,048-token limit. The **optimized** version used k=5 and the shorter-answer settings. I also tested k=5 alone, k=3 alone, and short answers alone with k=8, so I could see the effect of each change separately. I ran the optimized version a second time as well. In total, I made 180 API calls and used about 119,000 tokens, all on the free plan.

Results. The tables below show the average results per question across the 30-question test.

Table 1a. Average token use, savings and Groq processing time

Setting	Input	Reasoning	Visible answer	Total	Tokens saved	Cost saved	Groq (s)
Baseline (k=8)	701	122	40	864	–	–	0.23
k=5	494	137	42	673	22%	10%	0.23
k=3	358	133	36	526	39%	24%	0.24
k=8 + short answers	716	16	34	767	11%	32%	0.11
Optimized (k=5 + short)	**509**	**18**	**36**	**563**	**35%**	**46%**	**0.11**
Optimized, 2nd run	509	18	35	562	35%	47%	0.10

Table 1b. Answer quality

Setting	Correct /24	Right document cited /24	Unanswerable refused /6
Baseline (k=8)	24	24	6
k=5	23	24	6
k=3	21	22	6
k=8 + short answers	24	24	6
Optimized (k=5 + short)	**23**	**23**	**6**
Optimized, 2nd run	23	24	6

Token counts are the exact values returned by Groq. "Visible answer" means the output tokens minus the reasoning tokens. Cost was estimated using Groq's gpt-oss-20b prices: $0.075 per million input tokens and $0.30 per million output tokens [8]. "Groq (s)" is the time reported by Groq. The time I measured on my computer was about 4 to 6 seconds, mostly because of internet delays and rate-limit waiting. No answer was cut off in any setting.

What I learned.

- **The optimized agent passed my quality rule.** It used 35% fewer tokens per question (864 down to 563) and reduced the estimated cost by 46%. It refused all 6 questions that the documents could not answer, and no answer was cut off. It missed only one question, q21. The earlier retrieval check had already predicted this: with k=5, the section containing the VPN server address was not retrieved, so the model returned the VPN profile instead.
- **The biggest saving came from hidden reasoning.** I expected the three-sentence rule to make the visible answers much

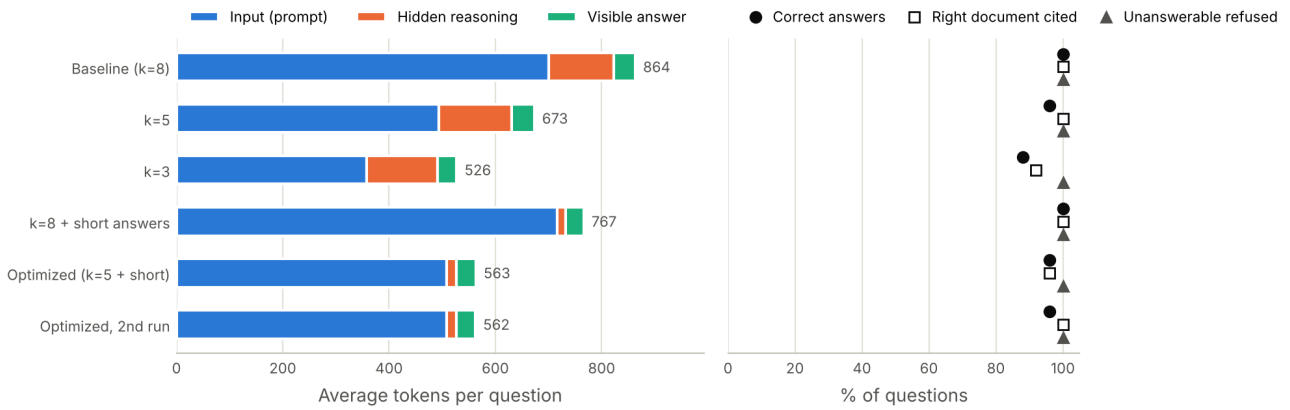


Figure 1: Token use (left) and answer quality (right) for each setting.

shorter, but gpt-oss-20b was already quite brief (40 to 34 visible tokens). Most of the saving came from reasoning, which dropped from 122 to 16 tokens per question. Groq's processing time was also cut by about half. Looking only at token totals would have been misleading: short answers alone saved 11% of tokens but 32% of the cost, while k=5 saved 22% of tokens but only 10% of the cost. This is because output tokens cost four times as much as input tokens.

- **Reducing k too much affected answer quality.** The free retrieval check was close to the results from the LLM: both expected and measured coverage were 100% with k=8 and 96% with k=5. With k=3, the check predicted 92%, while the measured score was 88%. The extra mistake with k=3 (q18) happened because the model left out "every Friday", not because search missed the information. For q14, where the required fact was missing, the model said it could not find the answer instead of making one up.
- **A second run is useful.** The two optimized runs got the same correctness score and missed the same question. However, one citation changed (23 compared with 24). With a test this small, a difference of one question should therefore be interpreted carefully.

Limits of this test. This was a small test with only 30 questions, so one question represents about 4 percentage points. The documents and questions were created with AI help and are cleaner than real company wiki pages. I tested only one model and only single questions, not long conversations. I checked answers by looking for keywords. I also reviewed the answers manually, but this method can still miss a correct answer that uses different wording. Finally, I selected k using the same questions I later used for evaluation. A fairer test would use a separate set of questions.

What I would try next.

1. **Test long conversations.** This would let me evaluate chat-history summaries (idea 3).
2. **Improve search.** I would try embeddings and a reranker to see whether a smaller k could still retrieve the right information.
3. **Add another evaluation method.** I would use an LLM-based evaluator, such as RAGAS [14], alongside the keyword check.
4. **Test another model.** Different models use hidden reasoning in different ways, so the results may not be the same.

D. References

[1] P. Lewis, E. Perez, A. Piktus et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. https://arxiv.org/abs/2005.11401

[2] N. F. Liu, K. Lin, J. Hewitt et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*, 12, 2024. https://arxiv.org/abs/2307.03172

[3] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, L. Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *EMNLP*, 2023. https://arxiv.org/abs/2310.05736

[4] F. Xu, W. Shi, E. Choi. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. *ICLR*, 2024. https://arxiv.org/abs/2310.04408

[5] Q. Wang, L. Ding, Y. Cao et al. Recursively Summarizing Enables Long-Term Dialogue Memory in Large Language Models. arXiv:2308.15022, 2023.

[6] Groq. Prompt caching (documentation). Accessed November 2026. https://console.groq.com/docs/prompt-caching

[7] L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.

[8] Morph. Groq pricing (October 2026): GroqCloud API rates per model. https://www.morphllm.com/groq-pricing

[9] Groq. Reasoning (documentation). Accessed November 2026. https://console.groq.com/docs/reasoning

[10] C. D. Manning, P. Raghavan, H. Schütze. *Introduction to Information Retrieval*, chapter 6. Cambridge University Press, 2008.

[11] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925, 2025.

[12] Groq. Rate limits (documentation). Accessed November 2026. https://console.groq.com/docs/rate-limits

[13] Groq. Model deprecations (documentation). Accessed November 2026. https://console.groq.com/docs/deprecations

[14] S. Es, J. James, L. Espinosa-Anke, S. Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *EACL (demos)*, 2024. https://arxiv.org/abs/2309.15217

How I used AI. I used Claude (Anthropic) as a support tool during this exercise. I first wrote the code and used Claude to help me identify and fix bugs along the way. I also used it to find research papers that inspired my approach and to prepare initial drafts of the fictional documents and test questions. I carried out the experiments using Groq API key, reviewed every answer whose score changed, and carefully went through the code to make sure I understood how each part worked. I also wrote the report and Claude helped me do it.